



iscte

INSTITUTO
UNIVERSITÁRIO
DE LISBOA

Advanced Computing

MPI Programming

Ricardo Fonseca

ricardo.fonseca@iscte-iul.pt

MPI programming

- **Distributed memory systems**

- MPI - message passing interface
- Point-to-point messages

- **Collective communications**

- One-to-all, all-to-one, all-to-all
- MPI global communications
- Performance considerations

- **Communicators**

- Partitioning parallel domain

- **Derived Datatypes**

- Complex datatype communication

- **Case study**

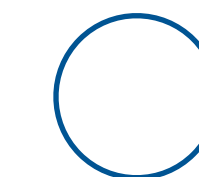
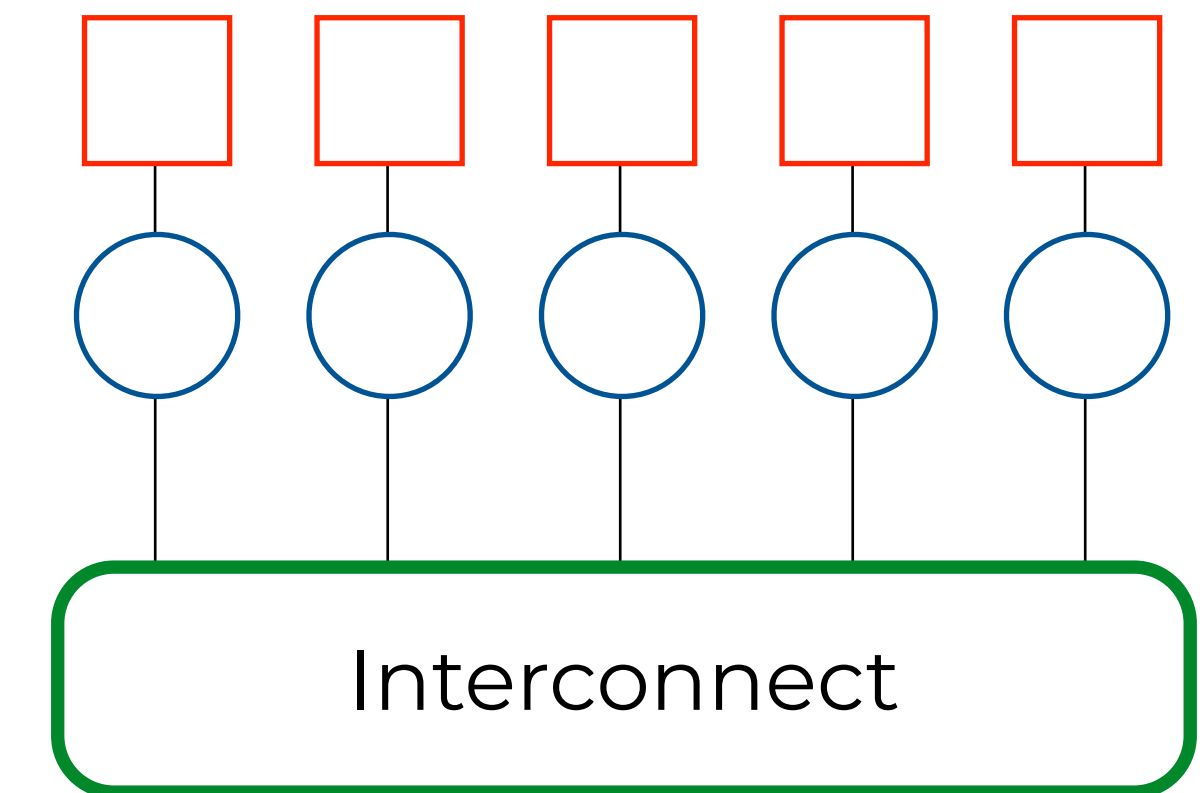
- Parallel Poisson equation



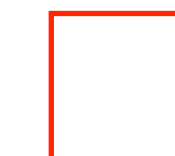
Review

Distributed memory systems

- **Distributed memory systems use separate address spaces for each processor**
 - Processing elements only have direct access to local data
- **Message Passing Interface (MPI) is the de facto standard for programming on distributed memory parallel computers**
 - Library of routines that enable message passing applications
 - Interface specification, not a concrete implementation
- **MPI implements a message Passing Model**
 - Each process has its own local address space
 - Processes communicate with each other through messages
- **MPI works with both distributed and shared memory**
 - Processes are logical, not physical
 - Extremely general/portable; even allows for heterogeneous systems



Process



Address Space



Point-to-point messages

- **MPI point to point messages represent the core of the library**

- Messages are described by source/destination task, datatype, size (of datatype elements) and tags
- Incoming messages may be selected based on source and/or tag
- Messages can be blocking or non-blocking, allowing us to overlap with computation or other messages if needed

- **Any communication pattern may be implemented using this type of messaging**

- Synchronization, one-to-many, many-to-one and many-to-many operations

- **However, manual implementation of these patterns is a complex task**

- Large programming burden
- Optimal performance requires advanced optimizations, matching algorithms and interconnect

blocking

`MPI_Send()`

`MPI_Recv()`

non-blocking

`MPI_Isend()`

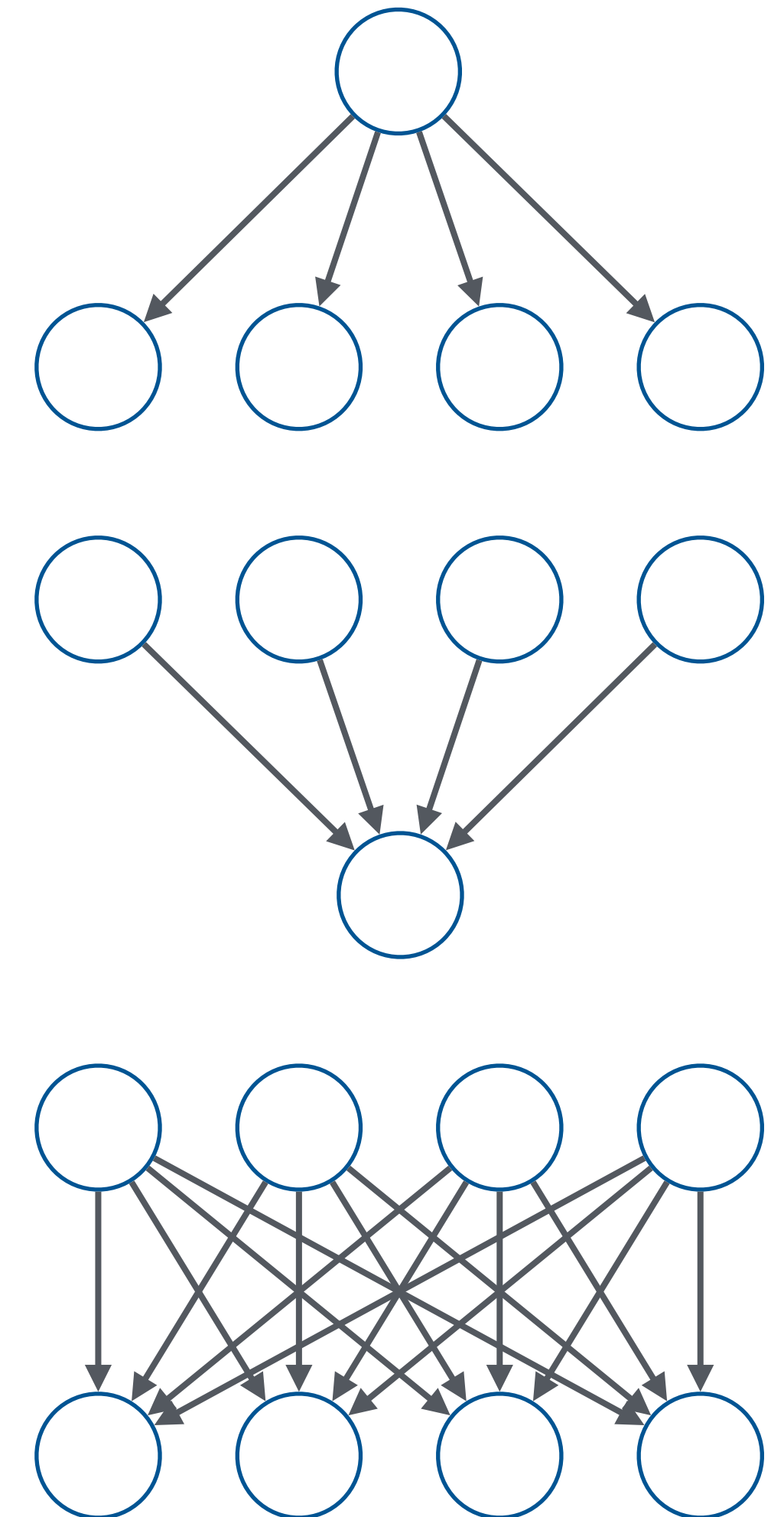
`MPI_Irecv()`

`MPI_Wait()`

Collective Communications

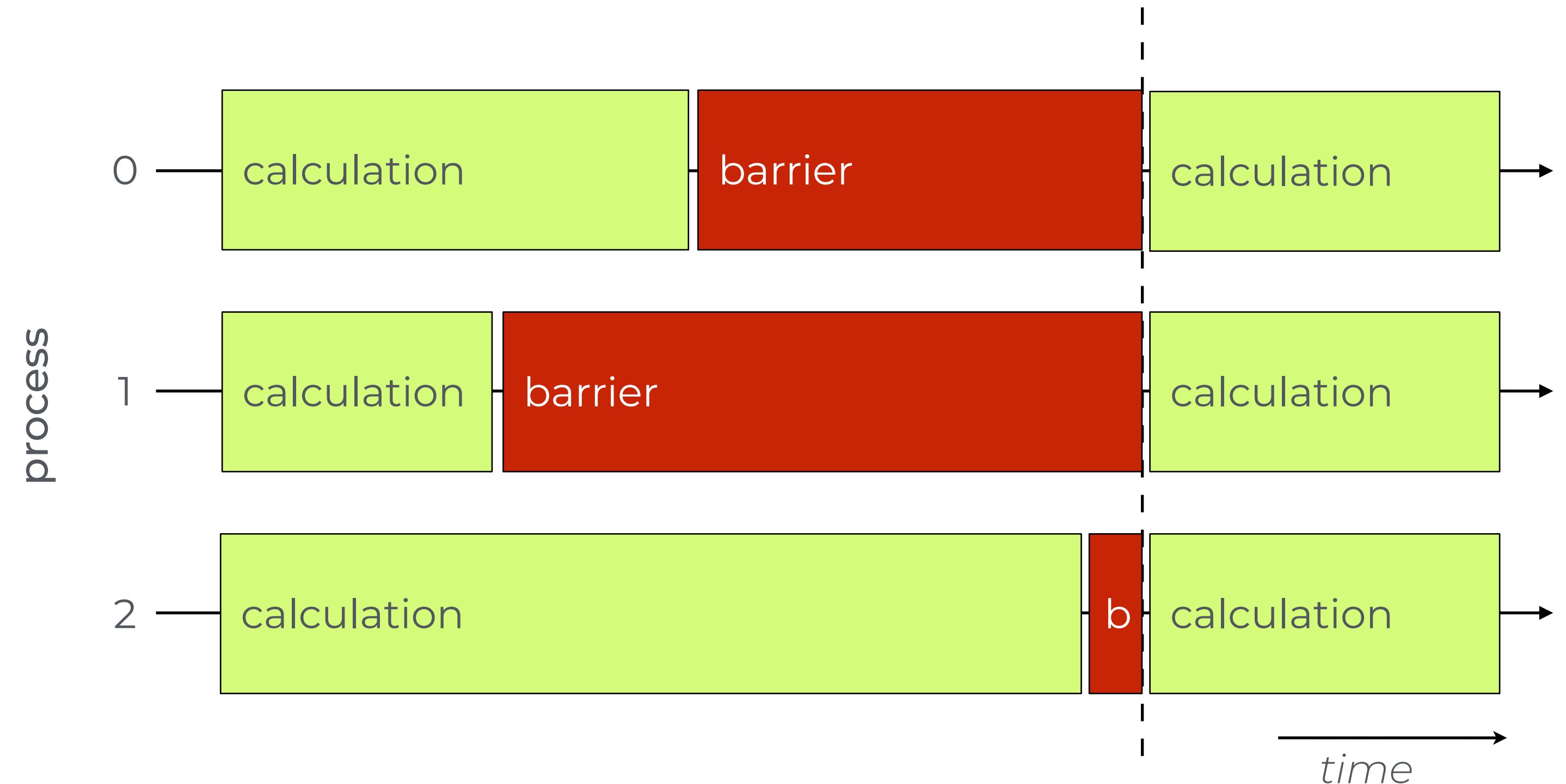
Collective communications

- **Many parallel problems require communication involving multiple processes**
 - These are referred to as collective communications
- **MPI also provides efficient interfaces for this type of communications**
 - Simplify parallel programming
 - Provide optimized versions
- **All processes in a communicator will be involved on collective communications**
 - Like in point-to-point communication, all processes involved are required to call the appropriate MPI function
- **All basic collective communication operations are blocking**
 - Functions will only return once the communication has completed
 - Implies a synchronization between all processes
 - *Note: recent MPI versions have introduced the concept of non-blocking collectives, we will not be covering those*



Synchronization

- **The simplest collective communication is a barrier operation**
 - The parallel program will not continue until all processes have reached this point
- **This is done using the `MPI_Barrier()` function**
 - No actual data exchange, only synchronization
- **All processes with the same communicator must call the function**
 - Otherwise the code will block

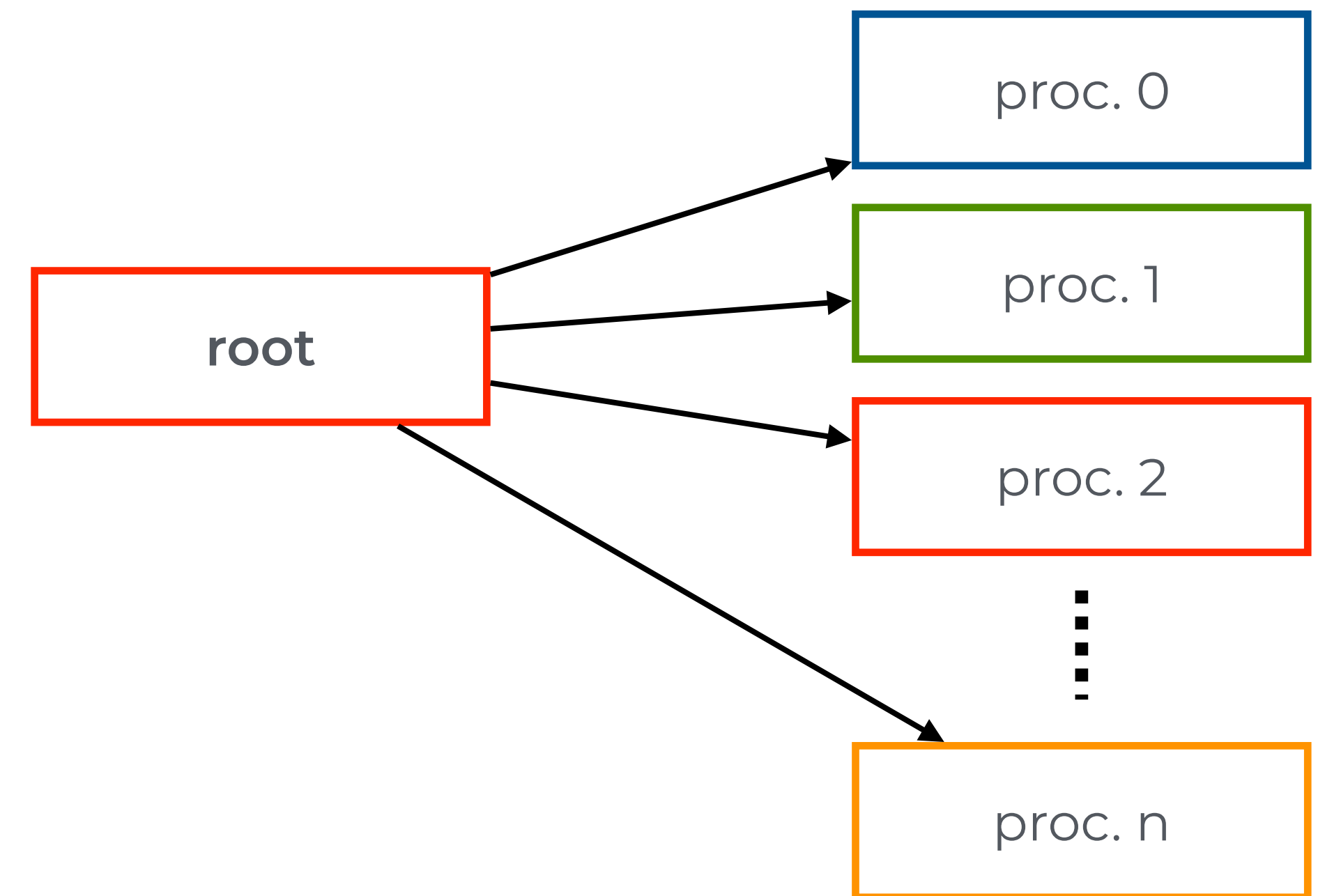


```
#include <mpi.h>

int MPI_Barrier(MPI_Comm)
```


Broadcast

- **The broadcast operation sends data from a root node to all other nodes**
 - Any node can be set as the root node for the operation
- **Use the `MPI_Bcast()` function**
 - As with other messaging functions we need to identify the datatype and the number of elements
 - No need to identify number of target ranks; broadcast involves all ranks in the communicator
- **This is also a blocking operation**
 - All processes will stop until the broadcast completes
 - As a result, processes will be synchronized



```
#include <mpi.h>

int MPI_Bcast(void *buffer,
              int count, MPI_Datatype datatype,
              int root, MPI_Comm comm)
```

Example

- **Simple broadcast example**

- Rank 0 initializes the data
- Broadcasts it to all ranks
- All ranks print rank number and value

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int value;
    if (rank == 0) value = 123;

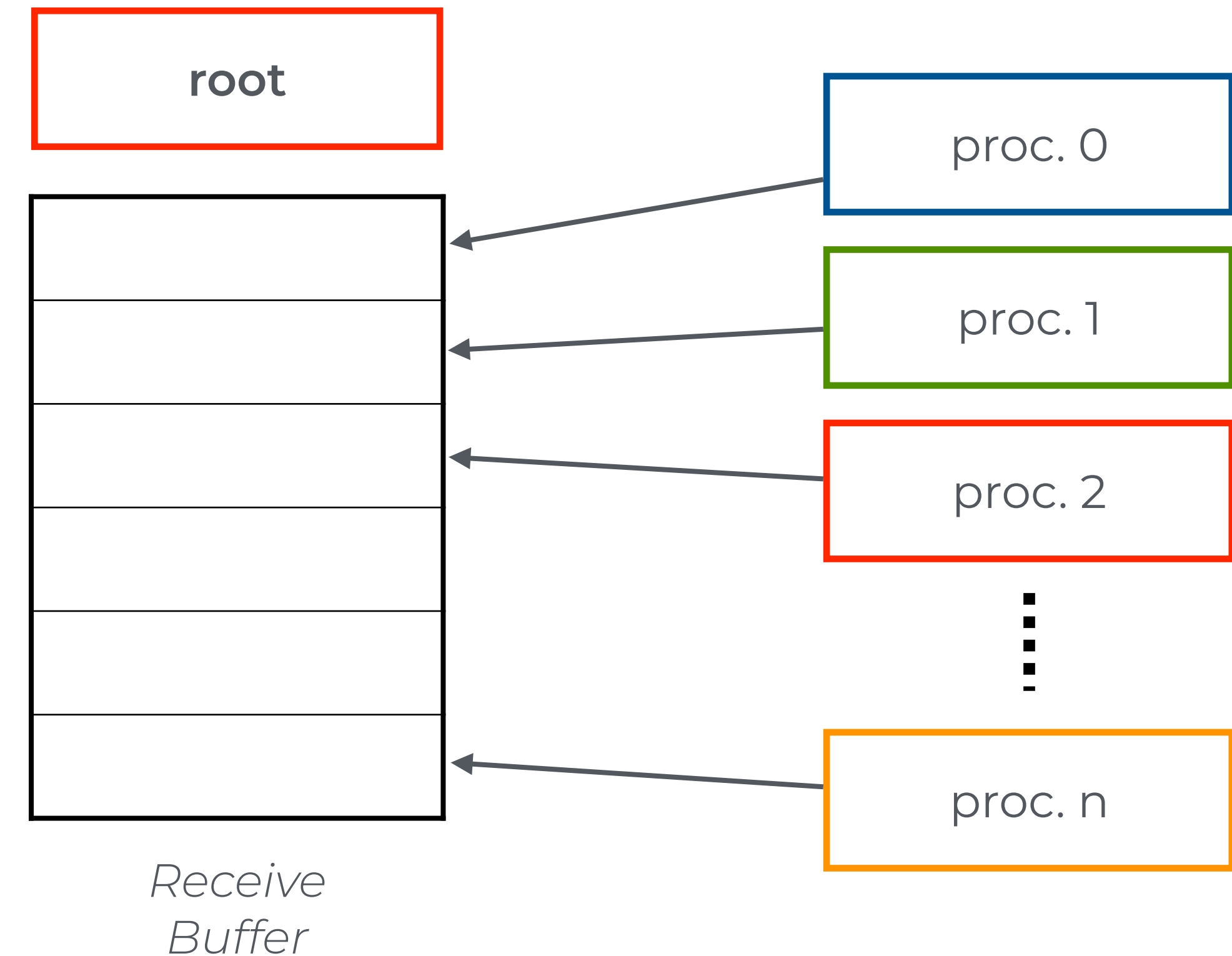
    MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);

    printf("Rank %d has value %d\n", rank, value);

    MPI_Finalize();
    return 0;
}
```


Gather

- **The gather operation gets data from all processes into a root process**
 - All processes send data, including root
 - Root process receives data and stores in rank order
- **Use the `MPI_Gather()` function**
 - Again, any process may be root
 - Root process receive buffer (recvbuf) must have enough space to hold all data
 - recvbuf is ignored for all non-root processes

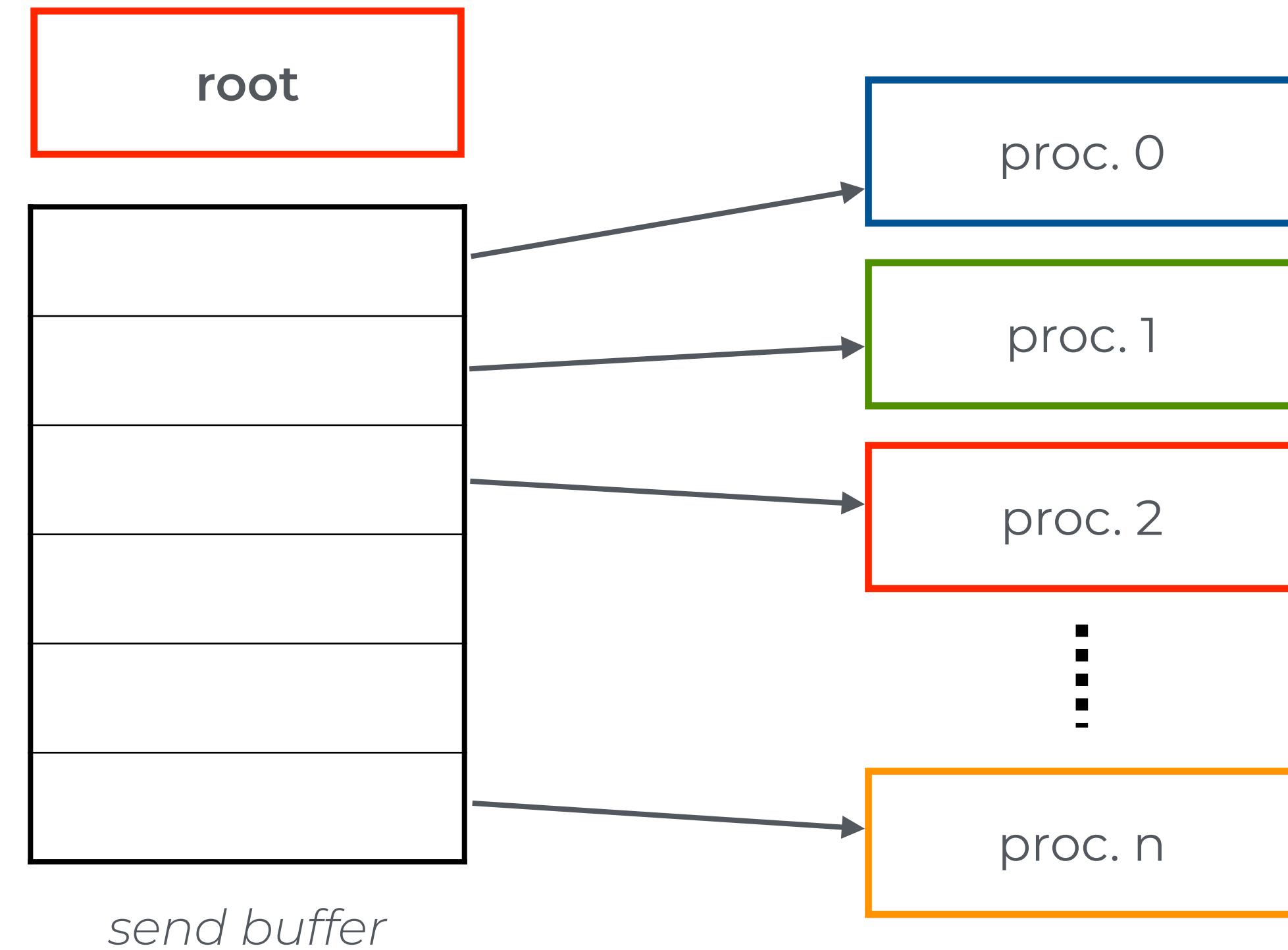


```
#include <mpi.h>

int MPI_Gather(
    const void *sendbuf, int sendcount, MPI_Datatype sendtype,
    void *recvbuf, int recvcount, MPI_Datatype recvtype,
    int root, MPI_Comm comm)
```

Scatter

- **The scatter operation sends data from a root process to all processes**
 - It works as the inverse of the gather operation
 - Data sent to each process is different
 - First rank gets the first chunk of the send buffer
 - Last rank gets the last chunk of the send buffer
 - This is different from a broadcast
- **Use the `MPI_Scatter()` function**
 - Root process also receives data
 - Just like in the gather operation, the root node is involved in both sending and receiving data



```
#include <mpi.h>

int MPI_Scatter(
    const void *sendbuf, int sendcount, MPI_Datatype sendtype,
    void *recvbuf, int recvcount, MPI_Datatype recvtype, int root,
    MPI_Comm comm)
```

Example

- **Scatter / Gather example**

- Rank 0 has array of values
- Scatters 1 element to each rank
- Each rank modifies data
- Gathers results from all ranks into rank 0 array

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int *data = NULL;
    int recv;

    if (rank == 0) {
        data = malloc(size * sizeof(int));
        for (int i = 0; i < size; i++) data[i] = i;
    }

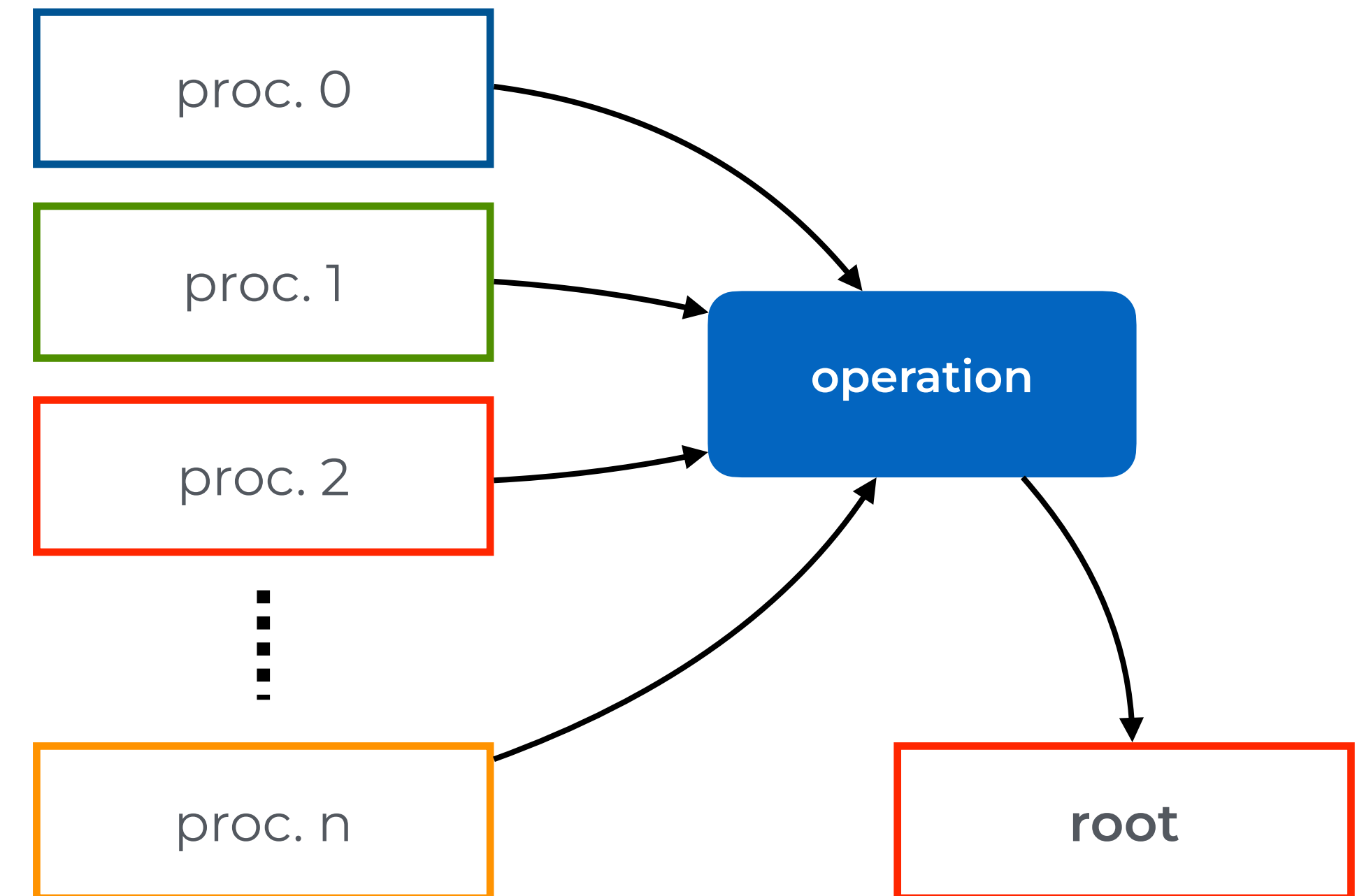
    MPI_Scatter(data, 1, MPI_INT, &recv, 1, MPI_INT, 0, MPI_COMM_WORLD);
    recv *= 2;
    MPI_Gather(&recv, 1, MPI_INT, data, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        for (int i = 0; i < size; i++)
            printf("%d ", data[i]);
        printf("\n");
        free(data);
    }

    MPI_Finalize();
    return 0;
}
```


Reduction

- **The reduction operation reduces values on all processes through a given operation**
 - This combines both communication and computation
- **Use the `MPI_Reduce()` function**
 - Results are returned on root node only
 - Use the `MPI_Allreduce()` function if you need the result on all the nodes
- **Supports many different operations**
 - Selected through the `op` parameter
 - MPI implements sum, max, min, prod and others
 - User defined operations are also supported
- **Works with scalars or arrays**
 - If inputs are arrays, outputs will be arrays of same size



```
#include <mpi.h>
```

```
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,  
               MPI_Datatype datatype, MPI_Op op, int root,  
               MPI_Comm comm)
```

Example

- **Reduce / Allreduce example**

- All processes initialize a different value
- Reduce command sums all values and returns on root node
- Allreduce command repeats sum returning result on all processes

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int local = rank + 1;
    int sum;

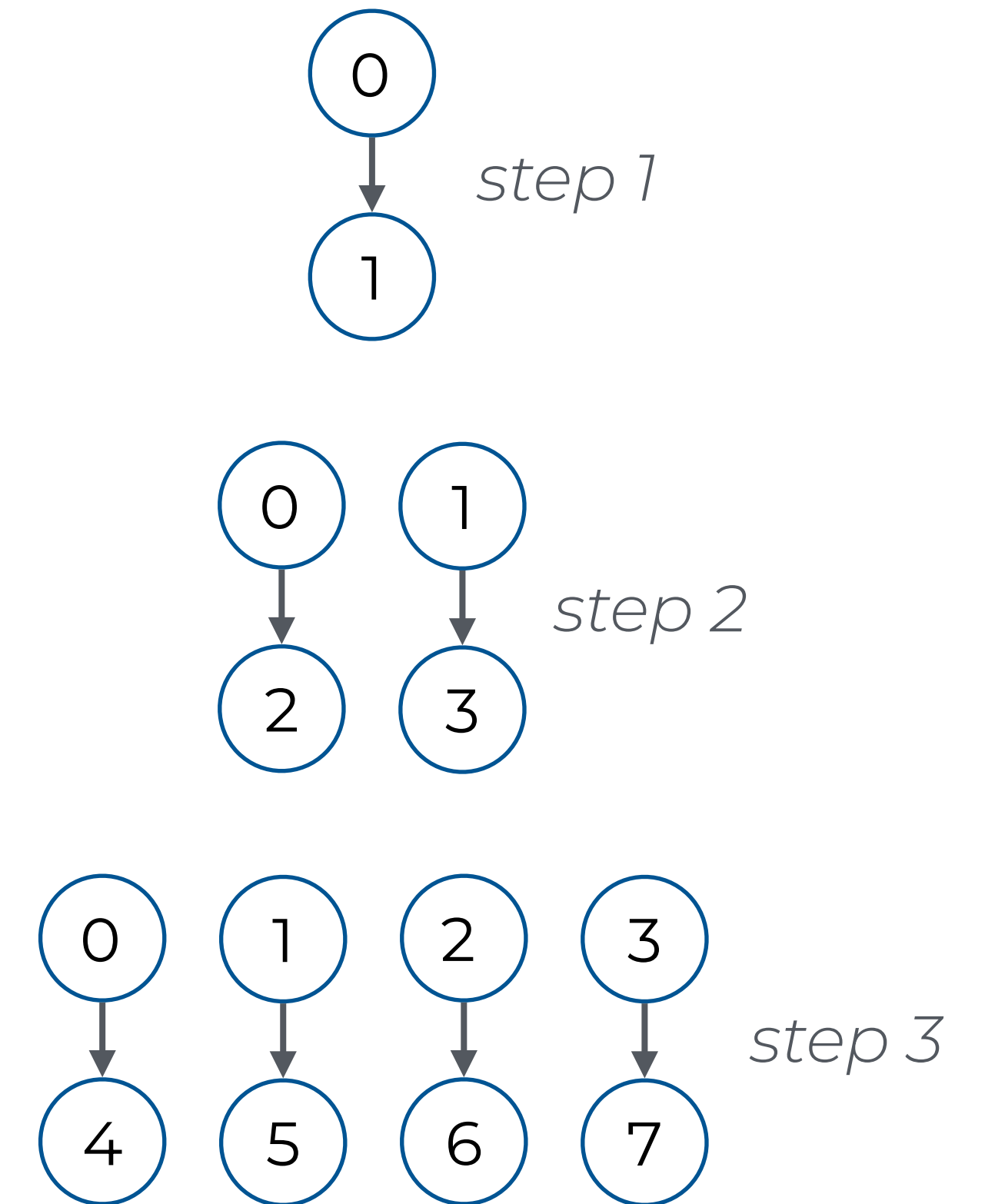
    MPI_Reduce(&local, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0) printf("Reduce sum = %d\n", sum);

    MPI_Allreduce(&local, &sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    printf("Rank %d sees sum %d\n", rank, sum);

    MPI_Finalize();
    return 0;
}
```

Performance implications

- **Consider a broadcast operation (MPI_Bcast)**
 - In a naïve (linear) algorithm implementation, root will send to every process
 - For P processes, this requires $P - 1$ steps, and a time $\mathcal{O}(P)$
- **A tree-based broadcast offers much better performance**
 - Root sends to 1 process; both forward to two more; etc.
 - Number of steps is reduced to $\log_2(P)$ and time to $\mathcal{O}(\log P)$
- **Most MPI collectives are implemented advanced optimizations**
 - Performance is generally very good
- **Prefer MPI collectives when possible**
- **Nevertheless, collective operations are implicit synchronization points**
 - Processes cannot complete the call until the communication has completed
 - Faster ranks will often wait for slower ones, which may lead to load imbalance issues
 - While $\log P$ scaling grows slowly, synchronization variation increases rapidly
 - A single slow rank affects the entire collective



broadcast example

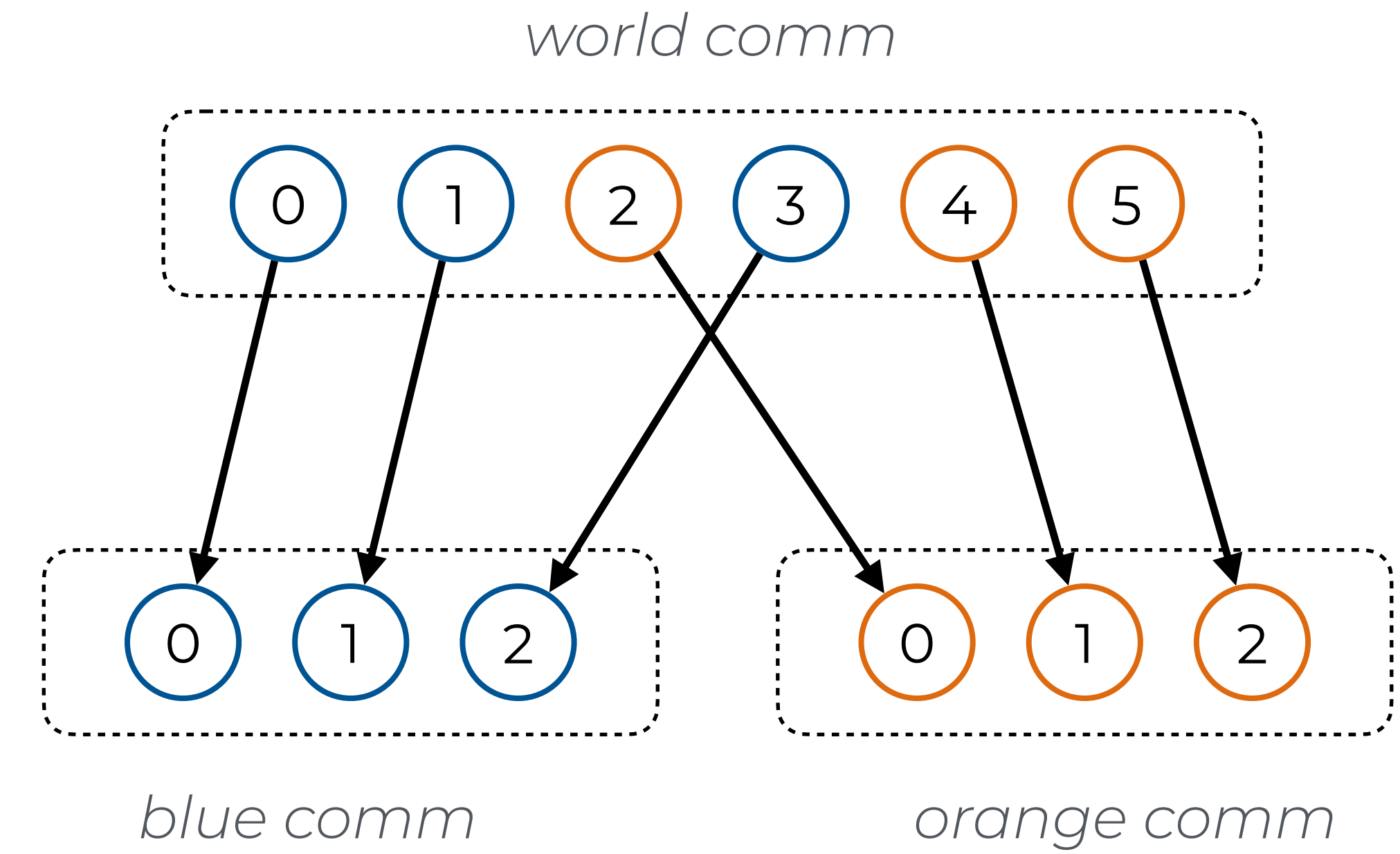
Communicators and groups

Communicators and Groups

- **So far we've only considered only global operations where every process communicates with everyone else**
 - MPI allows us to partition the communication space into independent spaces
 - Activities in these spaces may be independent from each other. For example, we may have a collective operation involving only a subset of the global parallel space
- **The two fundamental concepts that allow this separation are groups and communicators**
 - A **group** represents a set of processes that are ranked
 - A **communicator** consists of a group plus a communication context, enabling point-to-point and collective operations
- **Processes may belong to multiple communicators and groups**
 - A process rank exists only inside a communicator
 - The same process may have different ranks in different communicators
- **When launching an MPI program a global communicator is created**
 - MPI_COMM_WORLD combines a group of all the processes that were launched, plus the communication context that allows messaging

Creating communicators

- **We generally create new communicators by splitting an existing communicator**
 - Splitting a communicator is a collective operation inside it
 - Each process needs to specify a “color” identifying which of the new groups it will belong to
 - It may also choose a “key” that will control the rank inside the new group
- **This is achieved through the `MPI_Comm_split()` function**
 - The result is a set of independent sub communicators
 - Each process may have a totally different rank inside the new sub communicator
 - Using that sub communicator, a process will only be able to communicate with processes using the same sub communicator, that is processes that have the same color



```
#include <mpi.h>

int MPI_Comm_split(MPI_Comm comm, int color, int key,
                  MPI_Comm *newcomm)
```


Collectives and Resources

- **One of the main applications of communicators relate to collective operations**
 - Collectives are communicator-scoped
 - Processes outside a communicator are not in anyway involved in the operation
- **Using communicators allow us to use independent collectives**
 - Smaller communicators involve less synchronization and better performance
- **It also allows for modular algorithms**
 - Algorithm libraries may rely on communicators to coexist safely
 - By execution a library call with a separate communicator we isolate its communication space, preventing message interference
- **Remember that Communicators are resources**
 - Creating new communicators consumes internal MPI resources
 - Must be freed when no longer needed using `MPI_Comm_free()`

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int color = rank % 2;
    MPI_Comm newcomm;
    MPI_Comm_split(MPI_COMM_WORLD, color, rank,
                  &newcomm);

    int newrank;
    MPI_Comm_rank(newcomm, &newrank);

    printf("World rank %d → new rank %d (color %d)\n",
          rank, newrank, color);

    MPI_Comm_free(&newcomm);
    MPI_Finalize();
    return 0;
}
```

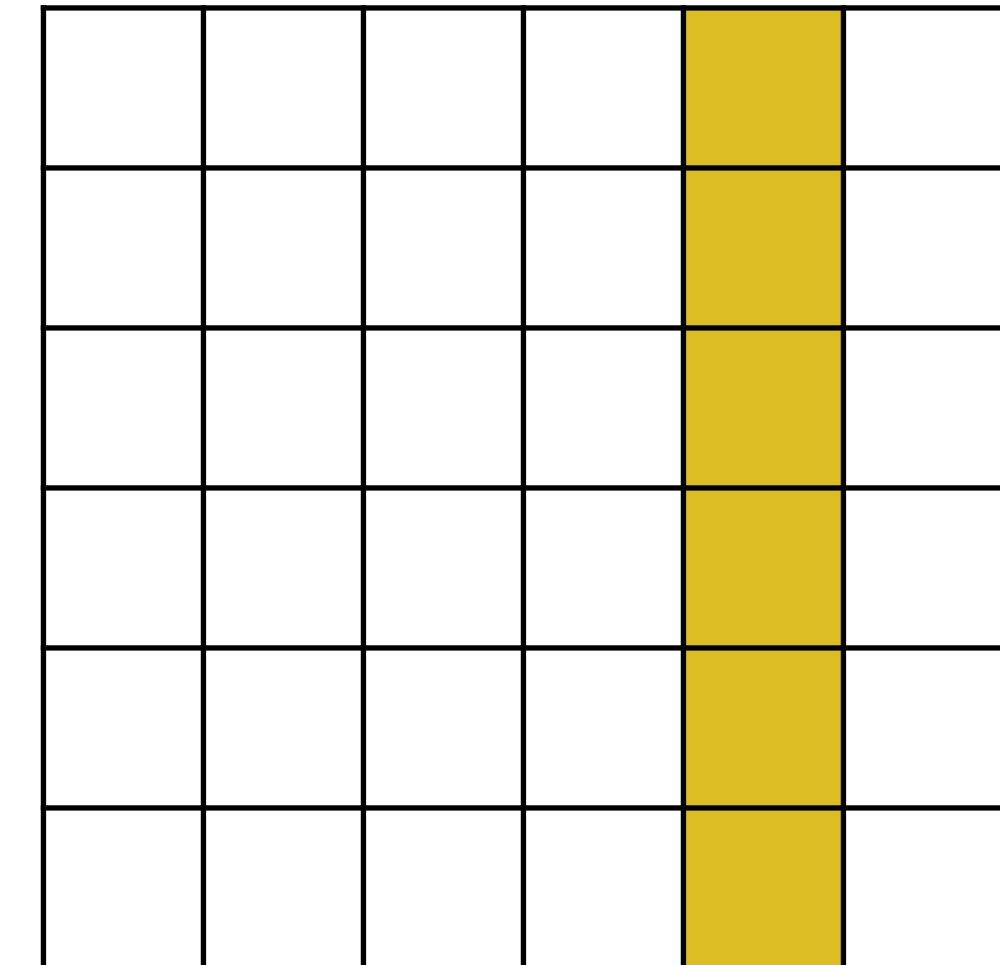
Derived datatypes

The need for derived datatypes

- **There are many situations where data is not simply a flat array**
 - For example, the data we need to send / receive may be non-contiguous or based on a struct
- **Manual packing/unpacking the data is not optimal**
 - It is error prone, hard to maintain and potentially slow
 - Requires additional buffers
- **MPI datatypes describe the memory layout directly**
 - It can be non-contiguous and use multiple existing datatypes
 - It does not allocate memory or move data
 - It is meant to be used in the communication routines

```
typedef struct {  
    float x,y;  
    int n;  
} t_message;
```

struct



non-contiguous data

Datatype lifecycle

- **The two most common datatype constructors are**
 - `MPI_Type_vector()` - Used for strided access, such as selecting matrix columns
 - `MPI_Type_create_struct()` - Used for mixed types / C structs (real-world data structures)
- **To use a derived datatype you must:**
 - Create the datatype using the multiple constructors available
 - Call `MPI_Type_commit()`. This will register your new datatype with the MPI environment and let you use it.
 - Use it in point-to-point or collective communications
 - If you don't need the datatype anymore delete it with `MPI_Type_free()`
- **Creating and committing a data-type is relatively expensive**
 - You should create it once and reuse it many times
 - Never build datatypes inside hot loops

```
#include <mpi.h>

int MPI_Type_vector(int count, int blocklength, int stride,
                   MPI_Datatype oldtype, MPI_Datatype *newtype)

int MPI_Type_commit(MPI_Datatype *datatype)

int MPI_Type_free(MPI_Datatype *datatype)
```


Example

- **Send a matrix column**
 - Matrix columns elements are not contiguous in memory
- **Use `MPI_Type_vector()` to construct datatype**
 - Specify memory stride of data
 - Based on `MPI_INT` datatype
- **Note that the datatypes on send / receive are different in this example**
 - The receive call just gets a simple array of `MPI_INT`
- **Real-world cases can mix and match**
 - We could also have used yet another derived datatype for receive

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int matrix[4][4];
    if (rank == 0)
        for (int i = 0; i < 4; i++)
            for (int j = 0; j < 4; j++)
                matrix[i][j] = i * 10 + j;

    MPI_Datatype column;
    MPI_Type_vector(4, 1, 4, MPI_INT, &column);
    MPI_Type_commit(&column);

    if (rank == 0)
        MPI_Send(&matrix[0][1], 1, column, 1, 0, MPI_COMM_WORLD);
    if (rank == 1) {
        int recv[4];
        MPI_Recv(recv, 4, MPI_INT, 0, 0, MPI_COMM_WORLD,
                 MPI_STATUS_IGNORE);

        for (int i = 0; i < 4; i++)
            printf("%d ", recv[i]);
        printf("\n");
    }

    MPI_Type_free(&column);
    MPI_Finalize();
    return 0;
}
```

Poisson problem

Poisson Problem

Interior

$$\nabla^2 u = f(x, y)$$

Boundary

$$u(x, y) = g(x, y)$$

Discretizing on a grid

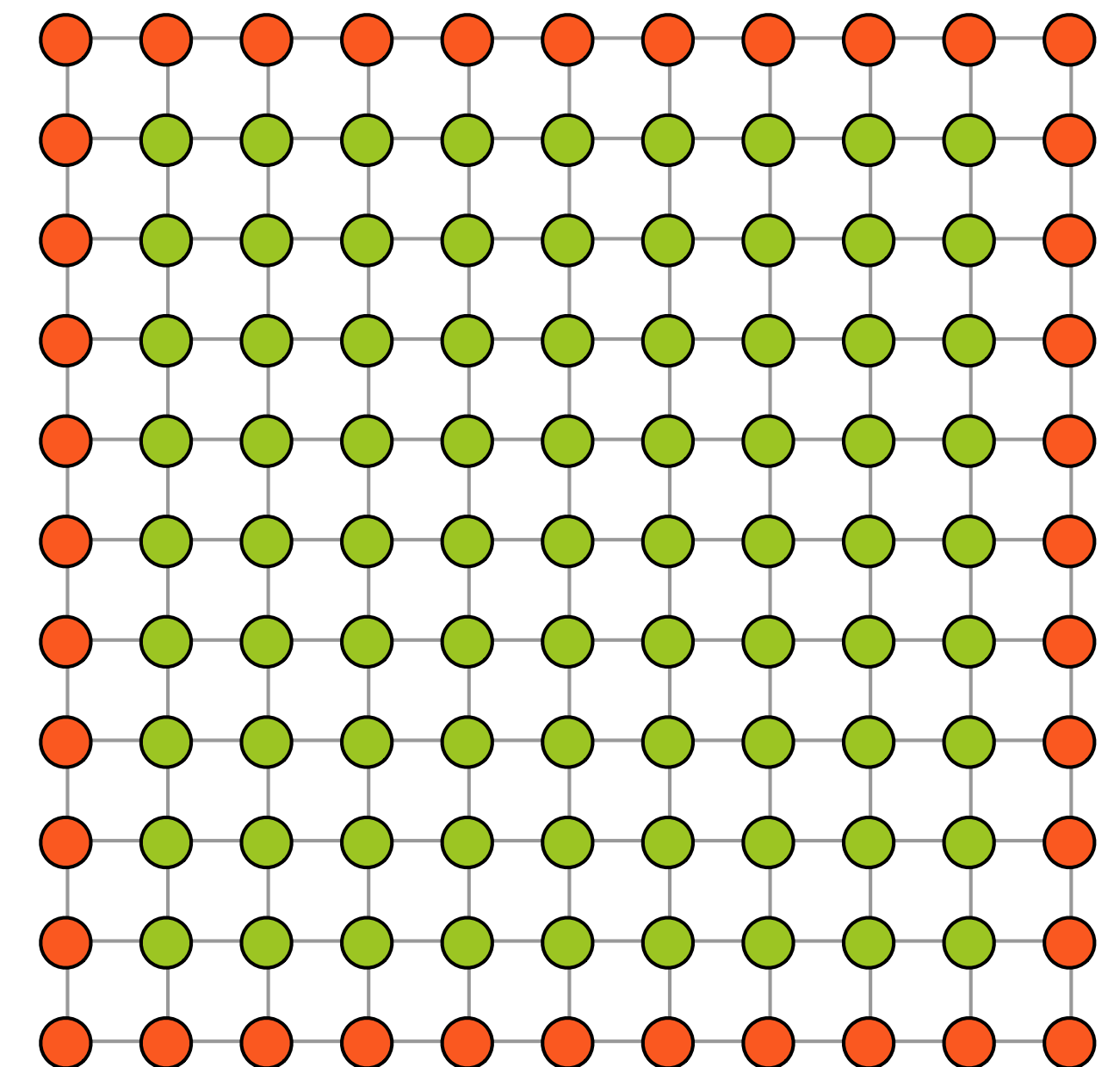
$$x_i = \frac{i}{n+1}, i = 0, \dots, n+1$$

$$y_j = \frac{j}{n+1}, j = 0, \dots, n+1$$

$$\frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2} = f_{i,j}$$

● $u_{i,j}$

● $g_{i,j}$



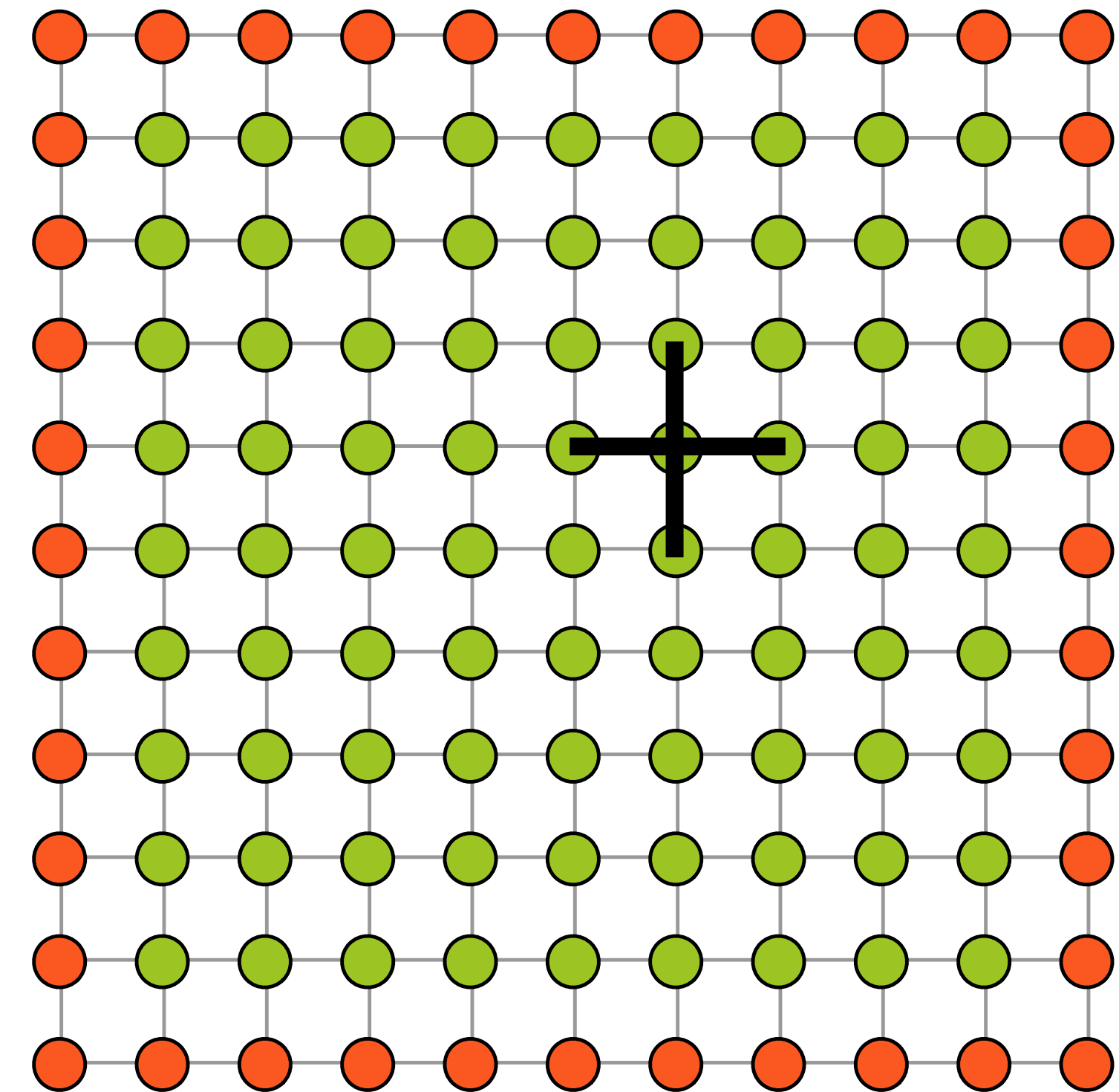
Jacobi Iteration

Rearrange finite difference equation

$$u_{i,j} = \frac{1}{4} \left(u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - h^2 f_{i,j} \right)$$

Estimate $u_{i,j}^{k+1}$ may be obtained from $u_{i,j}^k$

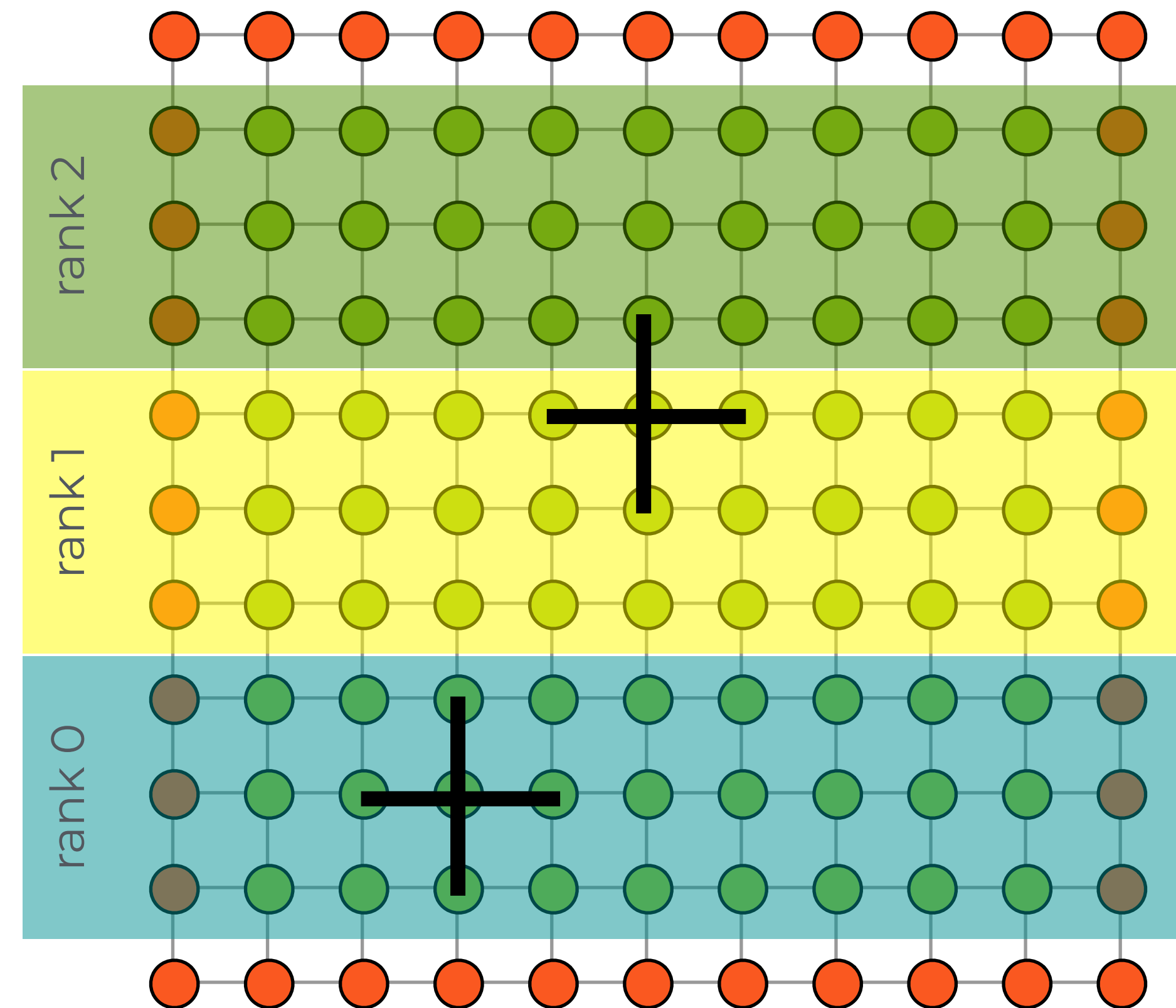
$$u_{i,j}^{k+1} = \frac{1}{4} \left(u_{i-1,j}^k + u_{i+1,j}^k + u_{i,j-1}^k + u_{i,j+1}^k - h^2 f_{i,j} \right)$$



To iterate on each point only nearest neighbors are required

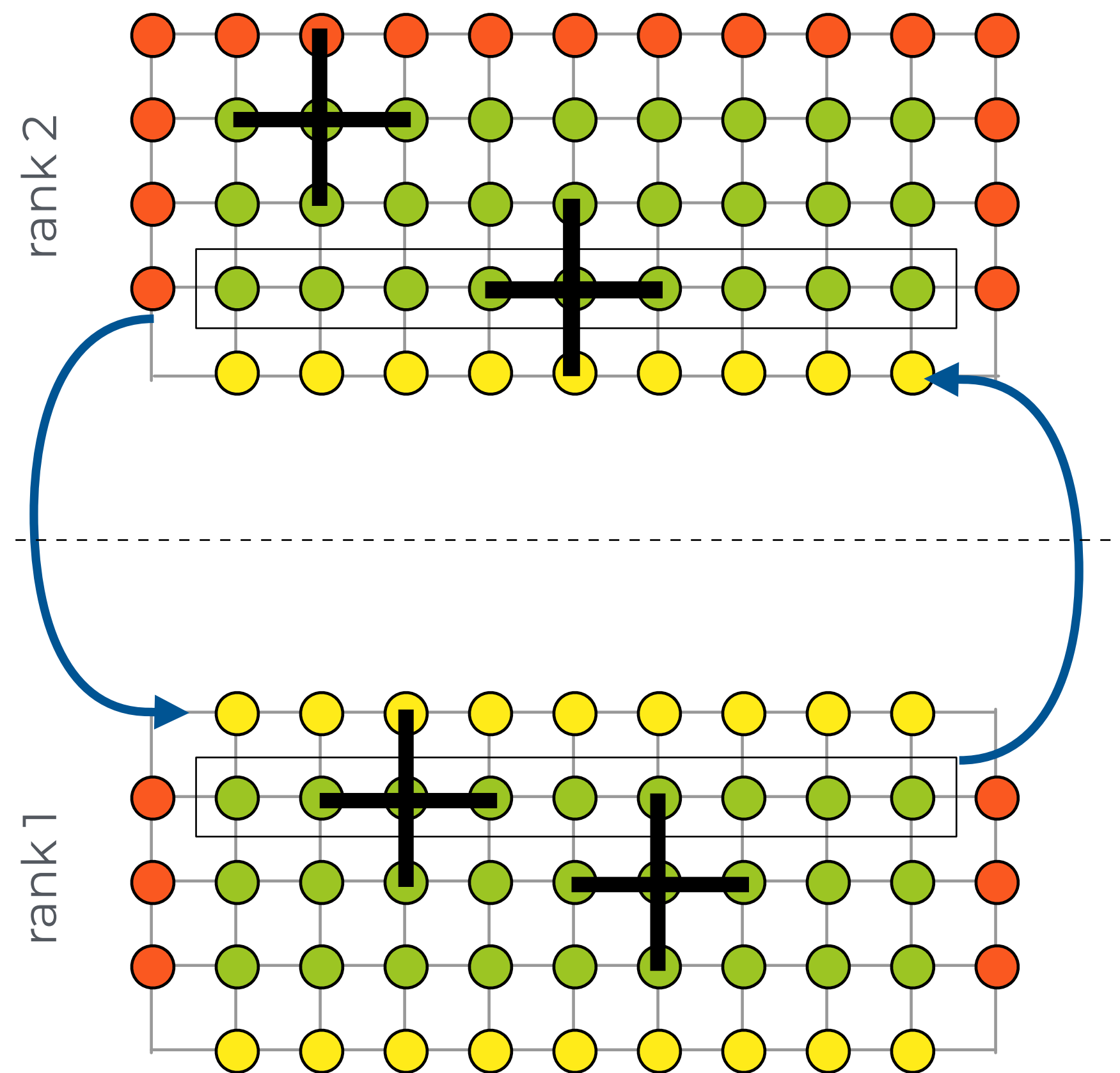
Parallelization

- **Use spatial domain decomposition**
 - Divide problem grid across available PEs
- **Most calculations use local data**
 - Use same implementation as serial code
- **Cells along edge are required from other nodes**
 - Information must be sent between neighbors



Use ghost/guard cells

Maintain local copy (●) of data
belonging to neighbor (●)



● Calculation Cells

● Guard Cells

Messages

- **Before computation copy values from neighbor to guard cells**
 - Exchange MPI messages
- **All data required for calculation is now available locally**
 - Guard cells can also be used for boundary condition values

Parallel Poisson Program

```
// Initialize MPI
MPI_Init( &argc, &argv );
int rank, size;
MPI_Comm_size( MPI_COMM_WORLD, &size );
MPI_Comm_rank( MPI_COMM_WORLD, &rank );

double *T      = malloc( (N/size+2)*(N+2)*sizeof(double));
double *Tnew = malloc( (N/size+2)*(N+2)*sizeof(double));

// ...

for( int iter = 0; iter < max_iter; iter++ ) {

    // Swap T / Tnew
    double *tmp = Tnew;
    Tnew = T; T = tmp;

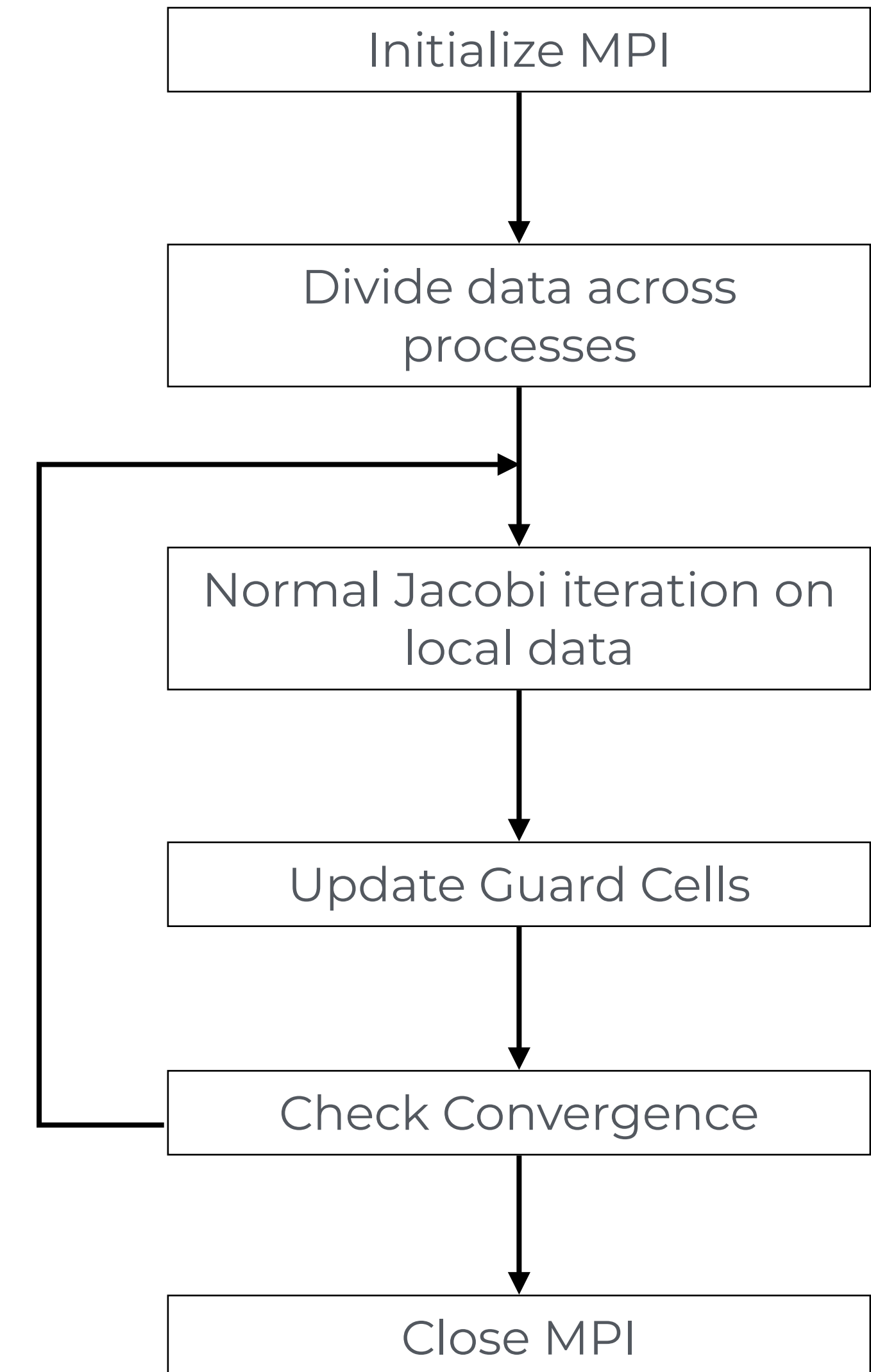
    // Jacobi iteration
    for( int j = 1; j <= N/size; j++ ) {
        for( int i = 1; i <= N; i++ ) {
            Tnew[ i + j*stride ] = 0.25 * ( T[ i+1 + j*stride ] + T[ i-1 + j*stride ] +
                                             T[ i + (j+1)*stride ] + T[ i + (j-1)*stride ] );
        }
    }

    // Update boundary values
    // ...

    // Check convergence
    // ...

}

// ...
// Close MPI
MPI_Finalize( );
```

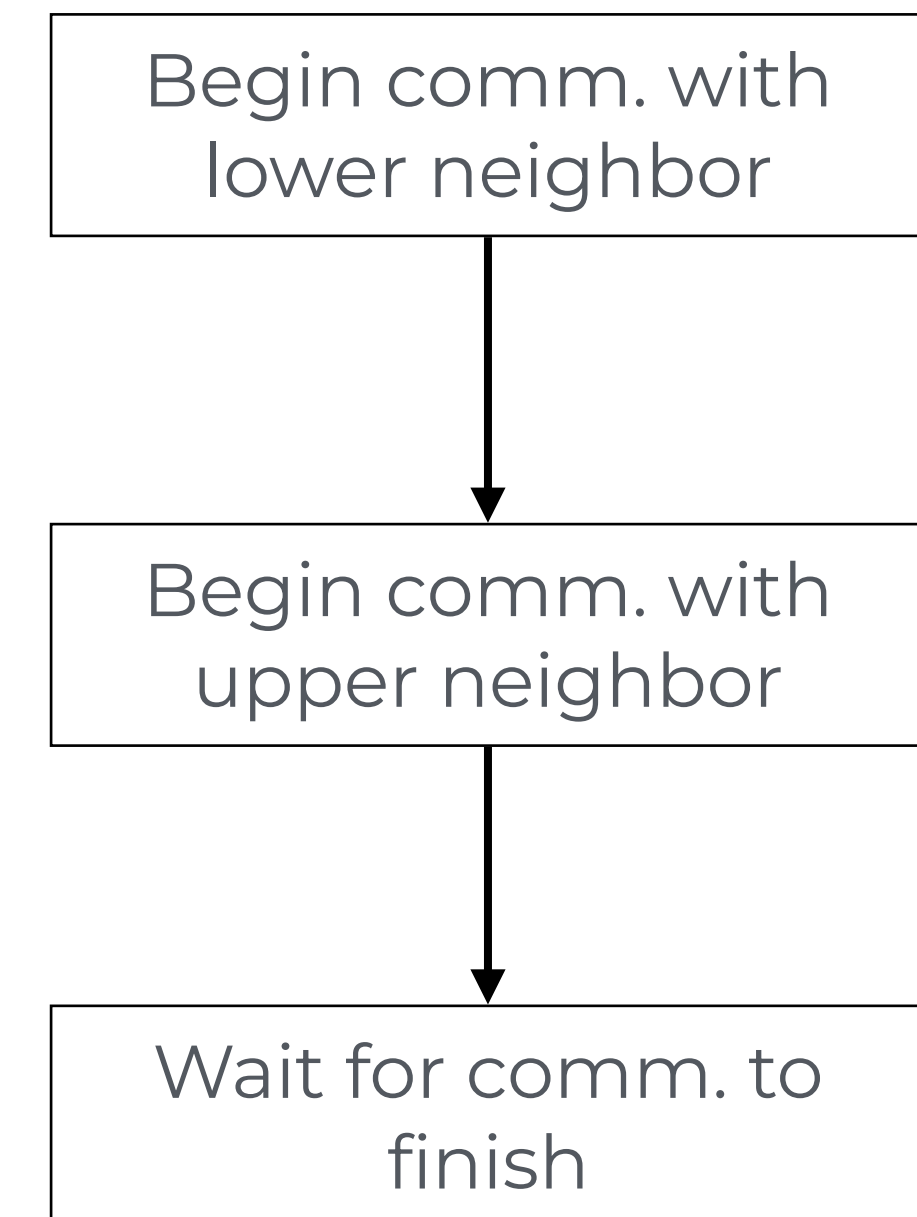


Updating Guard Cells

```
// Update boundary values
MPI_Request lsend, lrecv, usend, urecv;
int tag = 0;

// Communication with lower / upper neighbour
if ( rank > 0 ) {
    MPI_Irecv( &Tnew[ 1 + 0 * stride ], N, MPI_DOUBLE, rank-1, tag,
               MPI_COMM_WORLD, &lrecv );
    MPI_Isend( &Tnew[ 1 + 1 * stride ], N, MPI_DOUBLE, rank-1, tag,
               MPI_COMM_WORLD, &lsend );
}
if ( rank < size - 1 ) {
    MPI_Irecv( &Tnew[ 1 + (chunk+1) * stride ], N, MPI_DOUBLE, rank+1, tag,
               MPI_COMM_WORLD, &urecv );
    MPI_Isend( &Tnew[ 1 + chunk * stride ], N, MPI_DOUBLE, rank+1, tag,
               MPI_COMM_WORLD, &usend );
}

// Wait for messages to complete
if ( rank > 0 ) {
    MPI_Wait( &lrecv, MPI_STATUS_IGNORE );
    MPI_Wait( &lsend, MPI_STATUS_IGNORE );
}
if ( rank < size - 1 ) {
    MPI_Wait( &urecv, MPI_STATUS_IGNORE );
    MPI_Wait( &usend, MPI_STATUS_IGNORE );
}
```

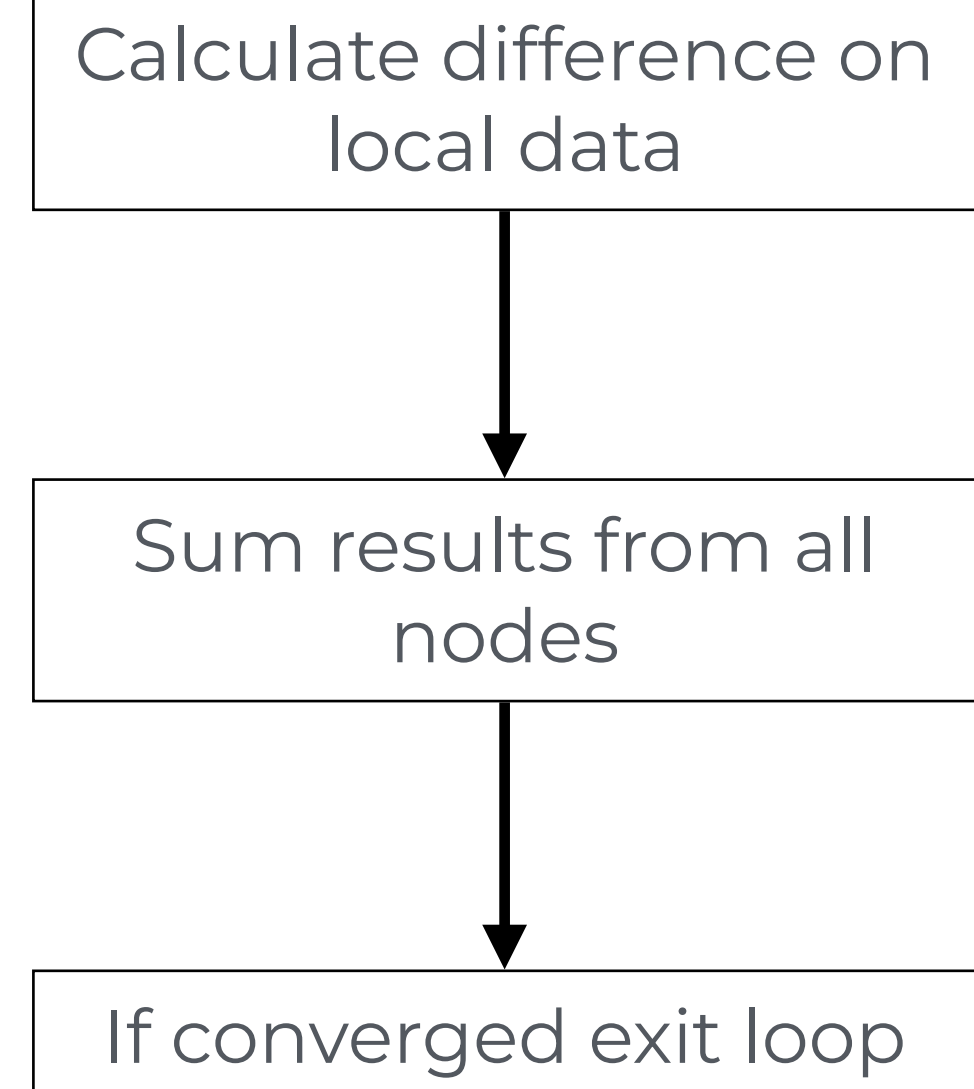


Calculating Convergence

```
// Check convergence
double eps = 0.0;
for( int j = 1; j <= chunk; j++) {
    for( int i = 1; i <= N; i++ ) {
        double delta = fabs( Tnew[ i + j*stride ] - T[ i + j*stride ] );
        if ( delta > eps ) eps = delta;
    }
}

double tmp;
MPI_Allreduce( &tmp, &eps, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD );

if ( tmp < max_eps ) break;
```



Overview

Overview

- **MPI provides a standard/portable toolset to program for distributed memory parallel computers**
 - Offers a complete library for implementing the message passing model
- **Point to point messages represent the core of the library**
 - Messages can be blocking or non-blocking, allowing us to overlap with computation or other messages if needed
- **MPI also supports collective communication**
 - High efficiency synchronization, one-to-many, many-to-one and many-to-many operations
 - This includes reduction operations, combining computation and communication
- **Parallel processes can be organized through the use of communicators**
 - This allow us to use independent collectives, and modular algorithms
 - MPI based libraries may operate in an isolated communication space
- **MPI datatypes provide a powerful tool for managing complex datatypes**
 - The feature extends supported datatypes beyond pre-defined types
 - Allow easy handling of non-contiguous types, avoiding manual packing / unpacking of data and additional buffers